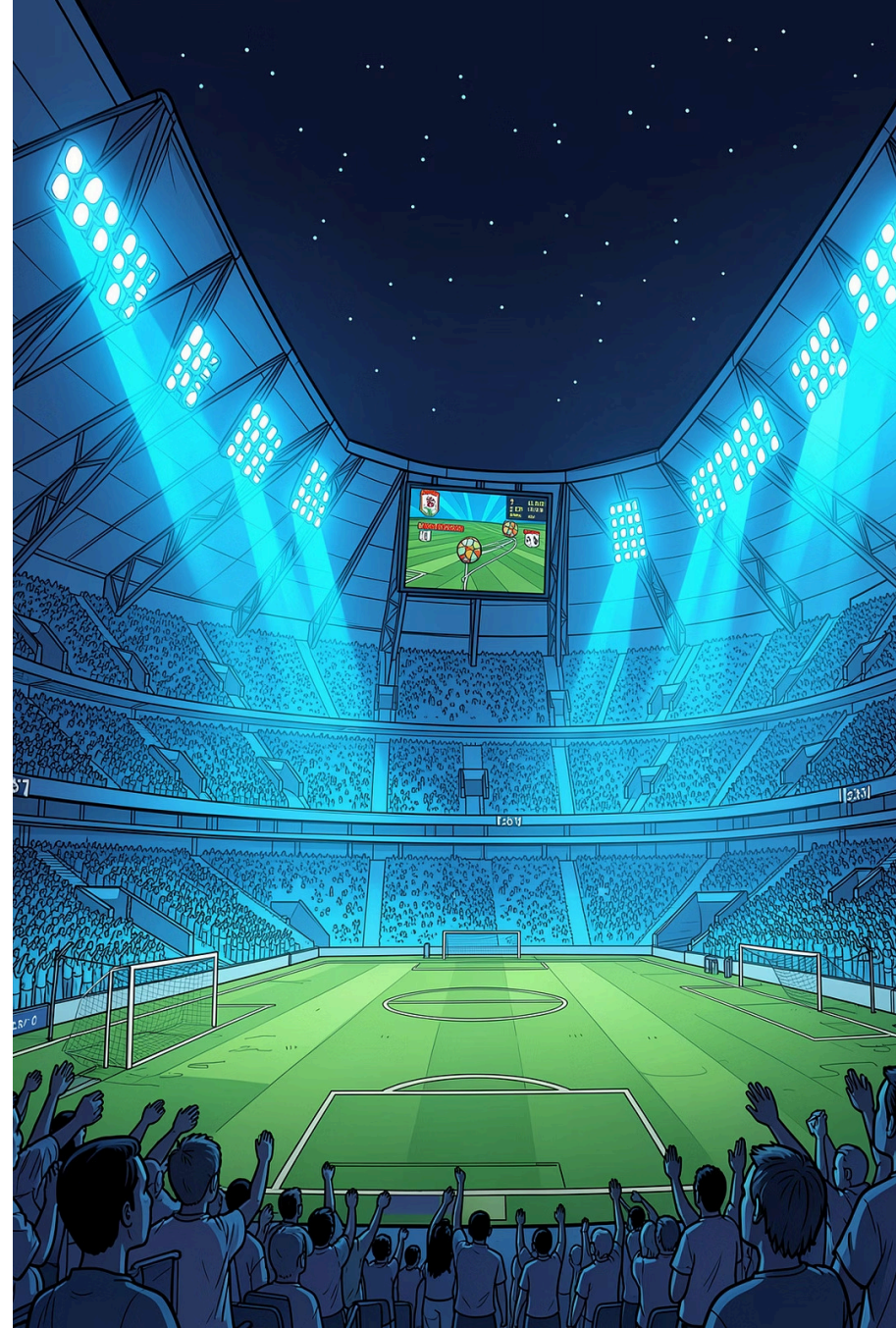


DraftKings

Gestión de Jugadores y Estadísticas de Fútbol

Aplicación híbrida multi-backend con **Angular/Ionic** · **Node.js** · **Spring Boot** · **CORBA**

CNSA · DAH · DWSC · TRWM



¿Qué es DraftKings?

App móvil híbrida para gestionar jugadores y estadísticas de fútbol. Permite crear, buscar, comentar y valorar jugadores, además de **generar equipos ideales con IA**.

Usuario No Registrado

Registrarse, ver listado y buscar jugadores

Usuario Registrado

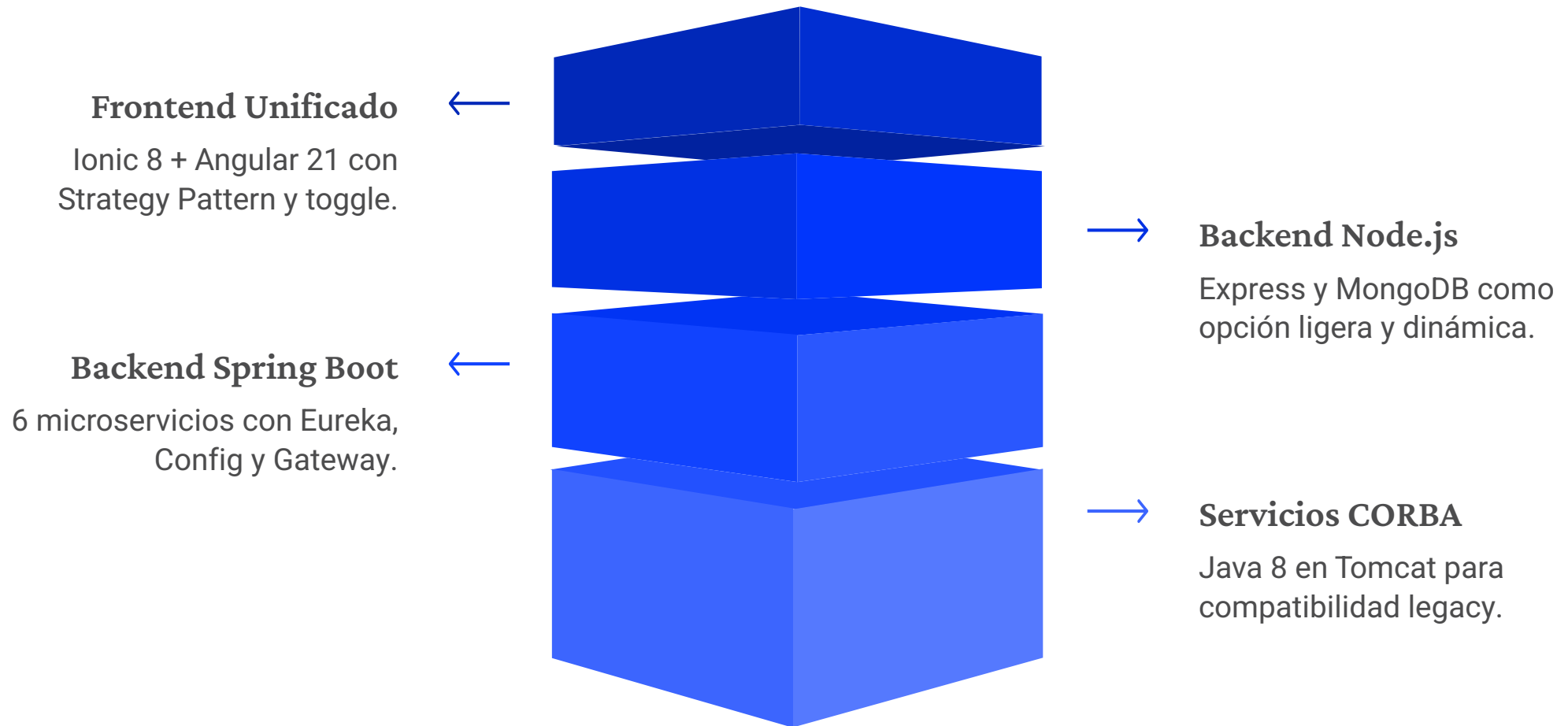
+ Importar jugadores, comentar, generar equipo ideal, ver noticias

Administrador

+ Editar/eliminar jugadores, borrar comentarios, publicar noticias

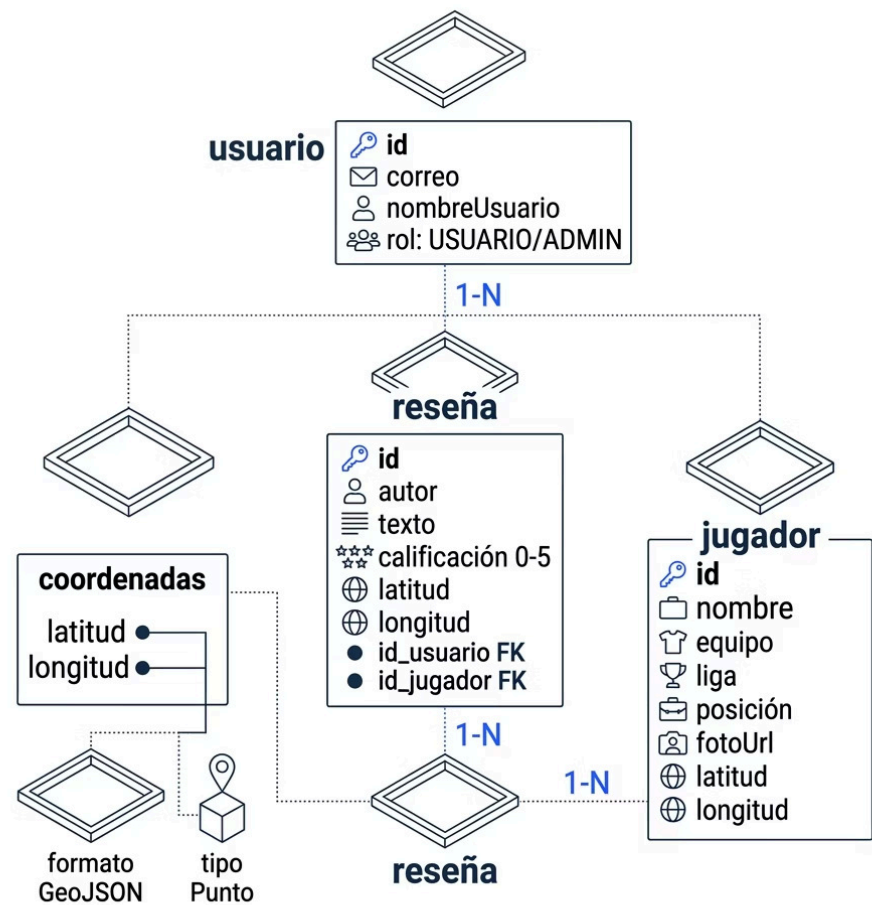
Arquitectura Multi-Backend

Frontend unificado que **conmuta entre dos backends independientes** en tiempo de ejecución mediante Strategy Pattern.



El frontend delega a servicios abstractos que se resuelven dinámicamente según el backend seleccionado.

Modelo de Datos



Entidades principales

- **users**: email, rol (USER/ADMIN), sincronización JIT
- **player**: datos personales, equipo, liga, posición, coordenadas
- **review**: texto, valoración 0–5★, geolocalización

❗ Las coordenadas se almacenan como **GeoJSON Point** con índice **2dsphere** en MongoDB.

ST-I

st API

blados

Tacke

frans

Roene

Elelope

Resgin

http://localhost:3000/

```

castrwmenial fot )
fhTTEE( "hentil("ite"; )
{
  hirnno hnleead(
  {
    HTTP
    HTTP: tate (:altH-3tlt");
    KB58: (ilactor= (crot="(tffc));
    HTTPU:rohhees);
  }
  HTTP Mont )
  HTTP Vcalhe Sseth"= (fi(Tesl))
  {
    winal hone )
    HTTP? batinp ("Iranontlel((..thintftyl")
  }
  HTTP fdond )
  HTTP Clodr= "prX(="("ltf");
  HvHID tassadfbitt);
  HTTP ("iben")
  }
  HTTP ("reompial rasaf"(L orpha)
  }
  HTTP )))

```

RESTful 12

DDcc.mnicatt||Qlinhzi,

X abauakm

fhirs h:rtalotc

Usuarios

2

Jugadores

3

Reviews

4

Noticias & IA

Noticias vía CORBA + `POST /api/tactics/recommendations`
(equipo ideal)

Códigos HTTP: **200/201/204** éxito · **400/401/403/404** error cliente · **500/503** error servidor

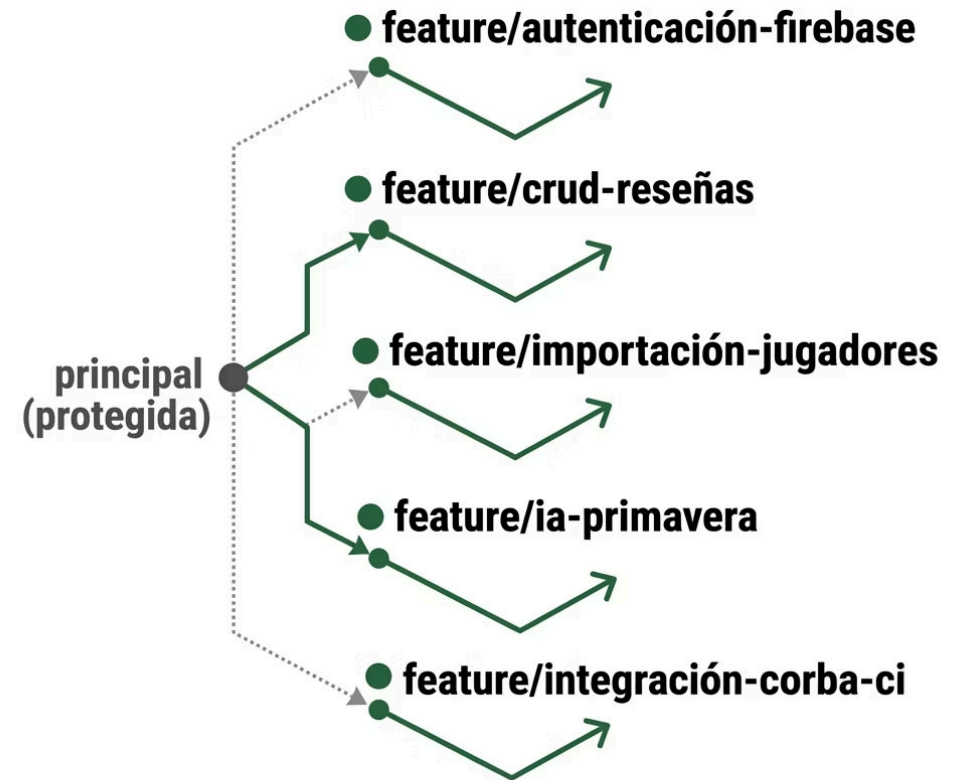
Organización del Repositorio Git

GitFlow simplificado

- **main** — producción (Protegida Actions) · Squash
- **dev** — integración continua (Protegida Actions) · Rebase
- **feature/*** — rebase sobre dev

Milestones (Issues)

- Fase 1: Setup · CI/CD
- Fase 2: Integración · Auth · APK
- Fase 3: Vistas · Endpoints
- Fase 4: CORBA · AI
- Fase 5: Tests · Docs



Patrones de Diseño y Decisiones Clave

Strateg / Factory

Cambio dinámico entre Node.js y Spring Boot sin modificar componentes según config de back.

Template Method

Flujos de autenticación con pasos comunes y hooks abstractos por backend.

Rollback

Compensación automática ante fallos en registro y creación de jugadores.

Circuit Breaker

Resiliencia con estados CLOSED, OPEN y HALF-OPEN y fallback seguro.

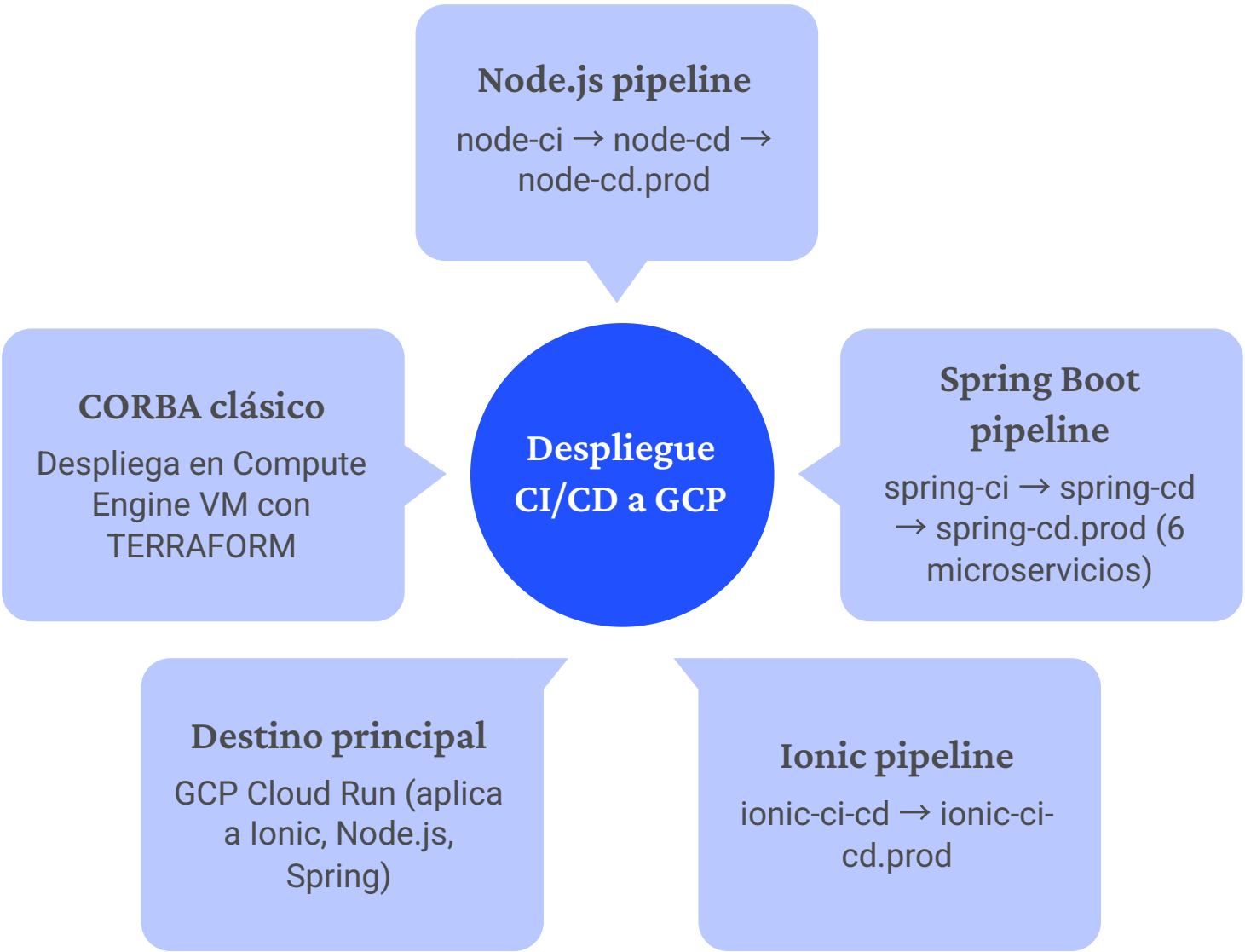
Observer

Angular Signals (signal, computed, effect) Reactividad sin mutación directa

Interceptor

AuthInterceptor (HTTP) Inyección automática de JWT en peticiones

CI/CD — 13 Workflows en GitHub Actions



Cada pipeline ejecuta tests, análisis estático y despliegue automático en GCP al superar los quality gates.

GCP Cloud Run Ionic · Node.js · 6 microservicios Spring	Compute Engine VM Servidor CORBA (Java 8 + Tomcat)
Artifact Registry Imágenes Docker versionadas para todos los servicios	APK Android Debug (dev) · Release firmado con apksigner (prod)

IA y Sistema Distribuido CORBA

Motor de IA — Equipo Ideal

Node.js: LangChain + Groq (gpt-oss-120b) → StructuredOutputParser + Zod

Spring Boot: Spring AI → PromptTemplate + BeanOutputConverter

 Entrada: posiciones del equipo. Salida: recomendaciones con nombre y justificación táctica.

01

Ionic → Spring Gateway (HTTP)

03

ORB Cliente  ORB Servidor (IIOP)

Cadena CORBA — Noticias

Circuit Breaker (Resilience4j): CLOSED → OPEN → HALF-OPEN con fallback diferenciado.

02

NewsServiceImpl → Servlet Bridge

04

Buffer IDL: 3 operaciones

Testing y Garantía de Calidad

180+

Tests totales

Unitarios, integración y E2E

80%

Cobertura Node.js

Jest + mongodb-memory-server

60%

Cobertura Spring

JaCoCo mínimo obligatorio

60

Tests Ionic

20 componentes + 40 E2E (Cypress)

Herramientas de calidad

- **ESLint** — style Angular/TypeScript
- **Checkstyle** — style Java
- **SonarQube** — bugs, vulnerabilidades, code smells
- **JaCoCo** — informe de cobertura integrado en pipeline

Cobertura Cypress E2E

- Autenticación (9 tests)
- CRUD Jugadores (13 tests)
- CRUD Reviews (12 tests)
- Importación + Navegación (7 tests)